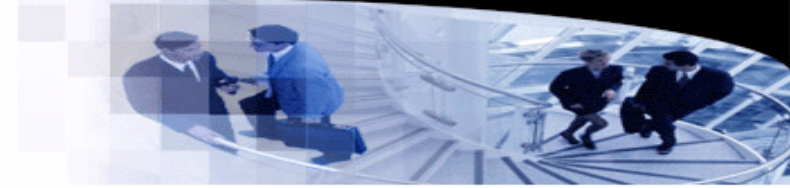




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

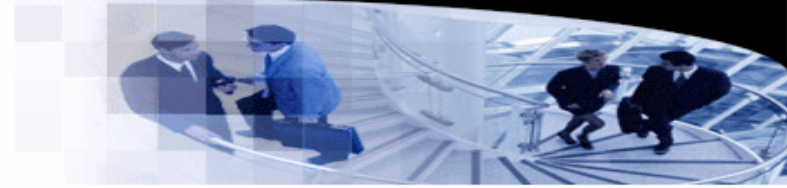
Lecture 10. Subtyping

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Motivation



- **Example**

- Let us write a program to manage a set of widgets in a UI window.
- The widgets may contain labels, edits, buttons or boxes.
- Each widget has a caption and a coordinate (x, y).
- Storing widgets in a single data structure would be convenient for us to apply operations on all widgets.
 - E.g., `let [] = List.iter (fun w->print_endline w.caption) ws.`
 - `let[] = List.iter (fun w->move w 10 -10) ws.`
- If we store the widgets in a list `ws`, what is the type of `ws`?

Motivation

- Example

- In OCaml, we can define a super class to represent shared behaviors of all widgets. Other classes maintain their specific properties.

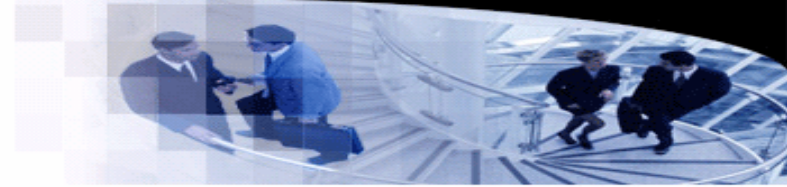
```
class widget =  
  object (self)  
    val mutable caption = ""  
    val mutable x = 0  
    val mutable y = 0  
    method move dx dy =  
      x <- x + dx;  
      y <- y + dy  
  end;;
```

```
class label =  
  object (self)  
    inherit widget as super  
    val mutable text = ""  
  end;;
```

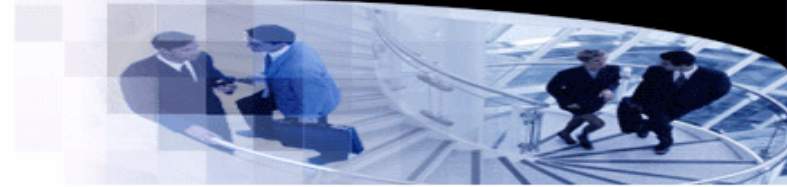
```
class edit =  
  object (self)  
    inherit widget as super  
    val mutable content = ""  
  end;;
```

- Now we can use a list to store different widgets.

```
# let ws =  
  let a = new label in  
  let b = new edit in  
  [a; b];;  
val ws : label list = [<obj>; <obj>]  
#
```



Motivation



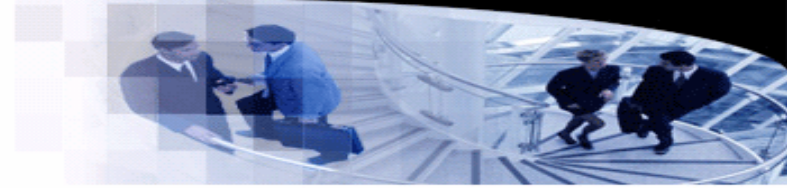
- Example

- Then we move all widgets in the list `ws`.

```
# let () =  
  List.iter (fun n -> n#move 10 10) ws;;  
#
```

- We know the widget class has a `move` method.
- Each object in the list inherits members from the widget class.
- The `move` operation can be **safely** applied to all objects in the list.
- That is one reason to introduce subtyping to programming languages.

Subtyping



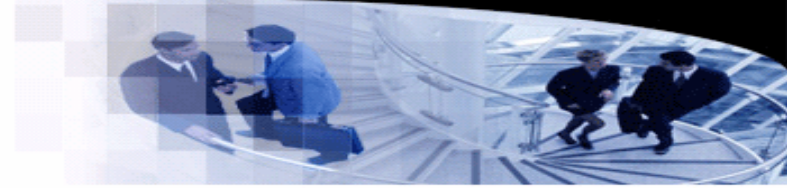
- Rules without subtyping

- Function applications have strict requirements to argument types.
- Consider the rule T-APP.

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \quad (\text{T-APP})$$

- The expression $(\lambda r : \{x : \text{Nat}\}. r.x) \{x = 0, y = 1\}$ is stuck.
- However, this application is reasonable, as $\{x = 0, y = 1\}$ contains more information than $\{x = 0\}$ and thus can be safely applied to this function.
- Subtyping rules are introduced to loosen these restrictions.
- Subtyping allows us to write more reusable functions.

Subtyping



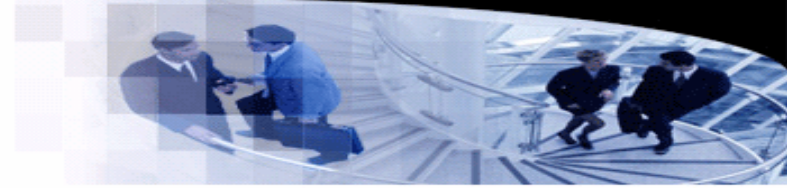
- Subsumption

- This rule tells us that, if $S <: T$, then every element t of S is also an element of T .

$$\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T} \quad (\text{T-SUB})$$

- Let us define a subtyping relation so that $\{x:\text{Nat}, y:\text{Nat}\} <: \{x:\text{Nat}\}$. (Is this relation reasonable?)
- We know $\vdash \{x = 0, y = 1\} : \{x:\text{Nat}, y:\text{Nat}\}$.
- Since $\{x:\text{Nat}, y:\text{Nat}\} <: \{x:\text{Nat}\}$, we have $\vdash \{x = 0, y = 1\} : \{x:\text{Nat}\}$ by rule T-SUB.
- Hence, $\{x = 0, y = 1\}$ can be applied to $\lambda r : \{x:\text{Nat}\}. r.x$.

Subtyping



- Subtype relation

- The subtype relation is formalized as a collection of inference rules for deriving statements of the form $S <: T$, pronounced “S is a subtype of T” (or “T is a supertype of S”).
- $S <: T$ means that each value of S can be safely used a value of T.

- Properties

- Reflexive

$$S <: S \quad (S\text{-REFL})$$

- Transitive

$$\frac{S <: U \quad U <: T}{S <: T} \quad (S\text{-TRANS})$$

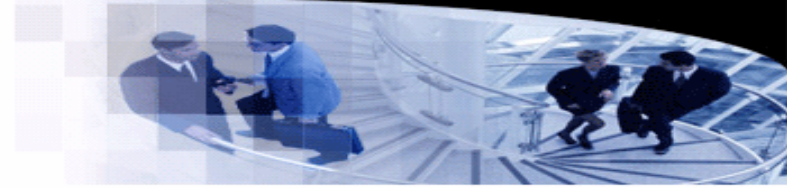
Subtyping

- Subtype relation
 - Records

$$\{\tau_i : T_i \mid i \in 1..n+k\} <: \{\tau_i : T_i \mid i \in 1..n\} \quad (\text{S-RCDWIDTH})$$

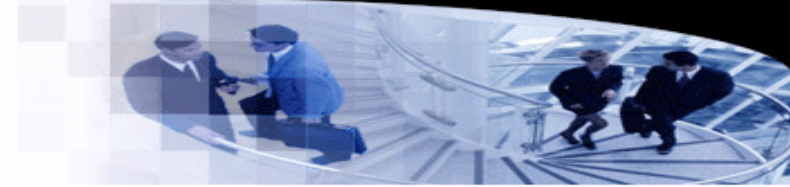
$$\frac{\text{for each } i \quad S_i <: T_i}{\{\tau_i : S_i \mid i \in 1..n\} <: \{\tau_i : T_i \mid i \in 1..n\}} \quad (\text{S-RCDDEPTH})$$

$$\frac{\{\tau_j : S_j \mid j \in 1..n\} \text{ is a permutation of } \{\tau_i : T_i \mid i \in 1..n\}}{\{\tau_j : S_j \mid j \in 1..n\} <: \{\tau_i : T_i \mid i \in 1..n\}} \quad (\text{S-RCDPERM})$$



Subtyping

- Subtype relation
- Examples

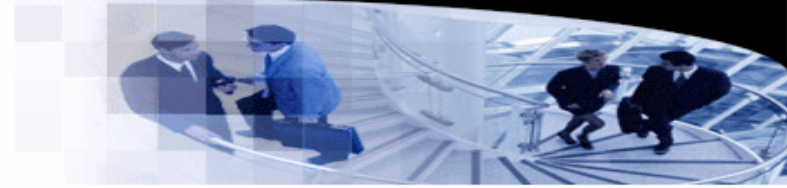


$$\begin{array}{c}
 \frac{}{\{a:\text{Nat}, b:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-RCDWIDTH} \qquad \frac{}{\{m:\text{Nat}\} <: \{\}} \text{S-RCDWIDTH} \\
 \hline
 \{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\{m:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}, y:\{\}\} \text{S-RCDDEPTH}
 \end{array}$$

$$\begin{array}{c}
 \frac{}{\{a:\text{Nat}, b:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-RCDWIDTH} \qquad \frac{}{\{m:\text{Nat}\} <: \{m:\text{Nat}\}} \text{S-REFL} \\
 \hline
 \{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\{m:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}, y:\{m:\text{Nat}\}\} \text{S-RCDDEPTH}
 \end{array}$$

$$\begin{array}{c}
 \frac{}{\{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\{m:\text{Nat}\}\} <: \{x:\{a:\text{Nat}, b:\text{Nat}\}\}} \text{S-RCDWIDTH} \qquad \frac{}{\{a:\text{Nat}, b:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-RCDWIDTH} \\
 \hline
 \frac{}{\{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\{m:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}\}} \text{S-RCDDEPTH} \qquad \frac{}{\{x:\{a:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}\}} \text{S-RCDDEPTH} \\
 \hline
 \{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\{m:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}\} \text{S-TRANS}
 \end{array}$$

Subtyping



- Subtype relation

- **Functions:** Specify under what circumstances it is safe to use a function of one type in a context where a different function type is expected.

$$\frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \quad (\text{S-ARROW})$$

- Given a function $f : S_1 \rightarrow S_2$, it accepts values of type S_1 .
- f also accepts values of type T_1 , as $T_1 <: S_1$ implies that each element in T_1 is also an element of S_1 .
- f returns values of type S_2 , which are also values of type T_2 (implied by $S_2 <: T_2$).
- f is also a function of type $T_1 \rightarrow T_2$.
- Therefore, $S_1 \rightarrow S_2 <: T_1 \rightarrow T_2$.

Subtyping

- Subtype relation

- Example

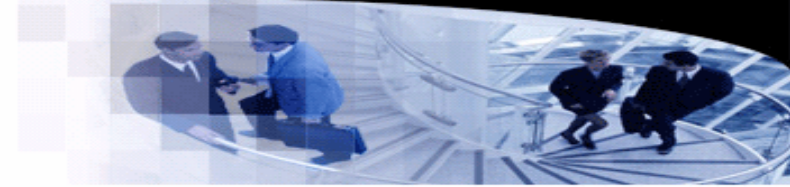
$$f \stackrel{\text{def}}{=} \lambda r:\{x:\text{Nat}\}. r.x$$

$$xy \stackrel{\text{def}}{=} \{x=0, y=1\}$$

$$Rx \stackrel{\text{def}}{=} \{x:\text{Nat}\}$$

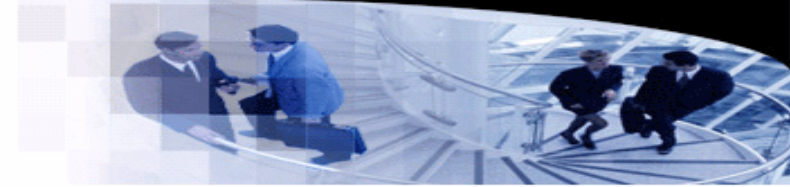
$$Rxy \stackrel{\text{def}}{=} \{x:\text{Nat}, y:\text{Nat}\}$$

$$\begin{array}{c}
 \vdots \\
 \hline
 \vdash f : Rx \rightarrow \text{Nat}
 \end{array}
 \quad
 \begin{array}{c}
 \hline
 \vdash 0 : \text{Nat} \quad \vdash 1 : \text{Nat} \\
 \hline
 \vdash xy : Rxy
 \end{array}
 \xrightarrow{\text{T-RCD}}
 \begin{array}{c}
 \hline
 \vdash xy : Rx
 \end{array}
 \quad
 \begin{array}{c}
 \hline
 Rxy <: Rx \\
 \hline
 \text{T-SUB}
 \end{array}
 \xrightarrow{\text{T-APP}}
 \vdash f xy : \text{Nat}$$



Subtyping

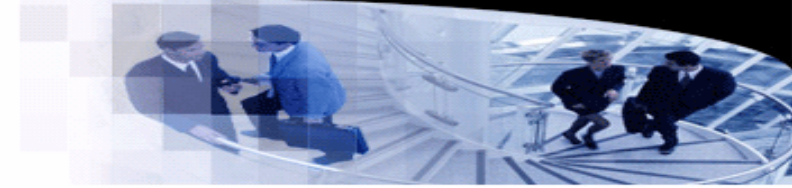
- Subtype relation
- Typing rules



→ <: Top		Based on λ _→ (9-1)	
Syntax		Subtyping	$\boxed{S <: T}$
$t ::=$	<i>terms:</i>	$S <: S$	(S-REFL)
x	<i>variable</i>	$\frac{S <: U \quad U <: T}{S <: T}$	(S-TRANS)
$\lambda x:T. t$	<i>abstraction</i>	$S <: \text{Top}$	(S-TOP)
$t \ t$	<i>application</i>	$\frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2}$	(S-ARROW)
$v ::=$	<i>values:</i>	Typing	$\boxed{\Gamma \vdash t : T}$
$\lambda x:T. t$	<i>abstraction value</i>	$\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$	(T-VAR)
$T ::=$	<i>types:</i>	$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$	(T-ABS)
Top	<i>maximum type</i>	$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 \ t_2 : T_{12}}$	(T-APP)
$T \rightarrow T$	<i>type of functions</i>	$\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T}$	(T-SUB)
$\Gamma ::=$	<i>contexts:</i>		
$\emptyset$	<i>empty context</i>		
$\Gamma, x:T$	<i>term variable binding</i>		
Evaluation	$\boxed{t \rightarrow t'}$		
$\frac{t_1 \rightarrow t'_1}{t_1 \ t_2 \rightarrow t'_1 \ t_2}$	(E-APP1)		
$\frac{t_2 \rightarrow t'_2}{v_1 \ t_2 \rightarrow v_1 \ t'_2}$	(E-APP2)		
$(\lambda x:T_{11}. t_{12}) \ v_2 \rightarrow [x \mapsto v_2] t_{12}$	(E-APPABS)		

Subtyping

- Subtype relation
- Typing rules



$\rightarrow \{\}$		Extends $\lambda_{\rightarrow}$ (9-1)	
New syntactic forms			
$t ::= \dots$	terms:	$\frac{t_1 \rightarrow t'_1}{t_1.l \rightarrow t'_1.l}$	(E-PROJ)
$\{l_i = t_i\}_{i \in 1..n}$	record		
$t.l$	projection	$\frac{t_j \rightarrow t'_j}{\{l_i = v_i\}_{i \in 1..j-1}, l_j = t_j, l_k = t_k\}_{k \in j+1..n} \rightarrow \{l_i = v_i\}_{i \in 1..j-1}, l_j = t'_j, l_k = t_k\}_{k \in j+1..n}}$	(E-RCD)
$v ::= \dots$	values:		
$\{l_i = v_i\}_{i \in 1..n}$	record value		
$T ::= \dots$	types:		
$\{l_i : T_i\}_{i \in 1..n}$	type of records		
New evaluation rules		New typing rules	
$\{l_i = v_i\}_{i \in 1..n}.l_j \rightarrow v_j$	$t \rightarrow t'$ (E-PROJ RCD)	$\Gamma \vdash t : T$	
		$\frac{\text{for each } i \quad \Gamma \vdash t_i : T_i}{\Gamma \vdash \{l_i = t_i\}_{i \in 1..n} : \{l_i : T_i\}_{i \in 1..n}}$	(T-RCD)
		$\frac{\Gamma \vdash t_l : \{l_i : T_i\}_{i \in 1..n}}{\Gamma \vdash t_l.l_j : T_j}$	(T-PROJ)

$\rightarrow \{\} <:$		Extends $\lambda_{<}$ (15-1) and simple record rules (15-2)	
New subtyping rules			
$\{l_i : T_i\}_{i \in 1..n+k} <: \{l_i : T_i\}_{i \in 1..n}$	$S <: T$ (S-RCDWIDTH)	$\{k_j : S_j\}_{j \in 1..n}$ is a permutation of $\{l_i : T_i\}_{i \in 1..n}$	
$\{l_i : S_i\}_{i \in 1..n} <: \{l_i : T_i\}_{i \in 1..n}$	(S-RCDDEPTH)	$\{k_j : S_j\}_{j \in 1..n} <: \{l_i : T_i\}_{i \in 1..n}$	(S-RCDPERM)

Subtyping

- Ascription via subtyping

- For simple types, ascriptions are actually **type annotations**.

→ as

Extends $\lambda_{\rightarrow}$ (9-1)

New syntactic forms

$t ::= \dots$
 $t \text{ as } T$

New evaluation rules

$v_1 \text{ as } T \rightarrow v_1$

$$\frac{t_1 \rightarrow t'_1}{t_1 \text{ as } T \rightarrow t'_1 \text{ as } T}$$

terms:
ascription

$t \rightarrow t'$

(E-ASCRIBE)

(E-ASCRIBE1)

New typing rules

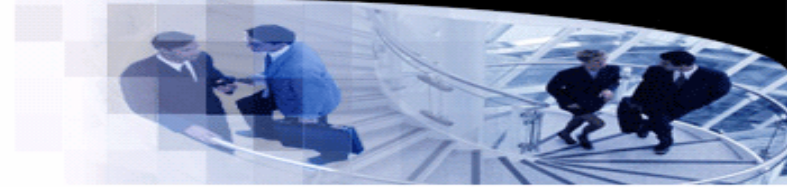
$$\frac{\Gamma \vdash t_1 : T}{\Gamma \vdash t_1 \text{ as } T : T}$$

$\Gamma \vdash t : T$

(T-ASCRIBE)

- However, ascriptions become **type castings** when they are used with subtyping.

Subtyping



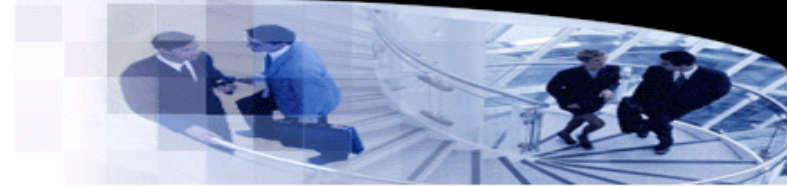
- **Ascription via subtyping**

- There are two forms of casting, i.e., **up-casts** and **down-casts**.
- Up-casts, in which a term is ascribed a supertype of the type that the type checker would naturally assign it, are instances of the standard ascription operator.
- We give a term t and a type T at which we intend to “view” t .

$$\frac{\Gamma \vdash t_1 : T}{\Gamma \vdash t_1 \text{ as } T : T} \quad (\text{T-ASCRIBE})$$

- Up-casts can be viewed as a form of **abstraction**.
- For example, if t is a record (or, more generally, an object), then we can use an up-cast to hide some of its fields (methods).

Subtyping



- **Ascription via subtyping**

- A down-cast allows us to assign types to terms that the type checker cannot derive statically.

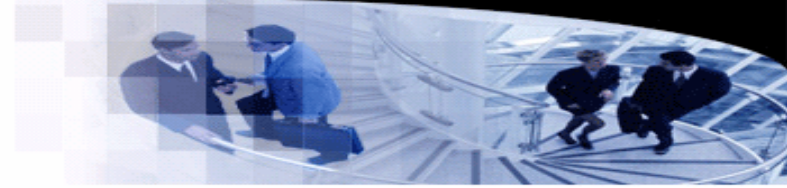
$$\frac{\Gamma \vdash t_1 : S}{\Gamma \vdash t_1 \text{ as } T : T} \quad (\text{T-DOWNCAST})$$

- Example
 - $f = \lambda(x:\text{Top}). (x \text{ as } \{a:\text{Nat}\}).a$
- Down-casts can be dangerous.
- We should verify the type before evaluation.

$$\frac{\vdash v_1 : T}{v_1 \text{ as } T \rightarrow v_1} \quad (\text{E-DOWNCAST})$$

- Down-casts provide simple dynamic polymorphisms without guarantee of safety.
 - Early version of Java collections. Java reflections.

Subtyping



- Intersection types

- A powerful refinement of the subtype relation can be obtained by adding an intersection operator to the language of types.
- Given T_1 and T_2 , the intersection type $T_1 \wedge T_2$ is the greatest lower bound of T_1 and T_2 .
- The intuition is captured by three new subtyping rules.

$$T_1 \wedge T_2 <: T_1 \quad (\text{S-INTER1})$$

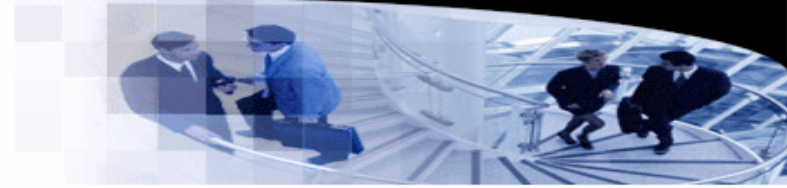
$$T_1 \wedge T_2 <: T_2 \quad (\text{S-INTER2})$$

$$\frac{S <: T_1 \quad S <: T_2}{S <: T_1 \wedge T_2} \quad (\text{S-INTER3})$$

- One additional rule allows a natural interaction between intersection and arrow types.

$$S \rightarrow T_1 \wedge S \rightarrow T_2 <: S \rightarrow (T_1 \wedge T_2) \quad (\text{S-INTER4})$$

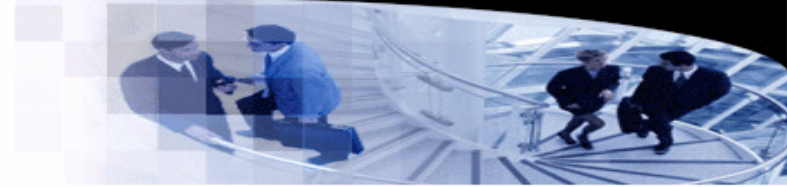
Subtyping



- Intersection types

- The interest of intersection types is that they support a form of **finitary overloading**.
- We might assign the type $(\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}) \wedge (\text{Float} \rightarrow \text{Float} \rightarrow \text{Float})$ to an addition operator that can be used on both natural numbers and floats.
- Tag bits can be used in the runtime representation of its arguments, for example, to select the correct instruction.
- Unfortunately, the power of intersection types raises some difficult pragmatic issues for language designers.
- A restricted form known as refinement types may prove more manageable.

Subtyping



- Intersection types

- Example in Typescript

```
class Egg { private kind: "Egg" }
class Milk { private kind: "Milk" }

//produces eggs
class Chicken { produce() { return new Egg(); } }

//produces milk
class Cow { produce() { return new Milk(); } }

//produces a random number
class RandomNumberGenerator { produce() { return Math.random(); } }

//requires an egg
function eatEgg(egg: Egg) {
    return "I ate an egg.";
}

//requires milk
function drinkMilk(milk: Milk) {
    return "I drank some milk.";
}
```

```
//given a chicken, produces an egg; given a cow, produces milk
let animalToFood: ((_: Chicken) => Egg) & ((_: Cow) => Milk) =
    function (animal: any) {
        return animal.produce();
    };
```

Subtyping

- Intersection types

- Example in Typescript

```
class Egg { private kind: "Egg" }
class Milk { private kind: "Milk" }

//produces eggs
class Chicken { produce() { return new Egg(); } }

//produces milk
class Cow { produce() { return new Milk(); } }

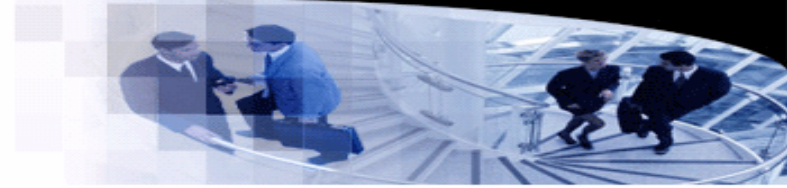
//produces a random number
class RandomNumberGenerator { produce() { return Math.random(); } }

//requires an egg
function eatEgg(egg: Egg) {
    return "I ate an egg.";
}

//requires milk
function drinkMilk(milk: Milk) {
    return "I drank some milk.";
}
```

```
//given a chicken, produces an egg; given a cow, produces milk
let animalToFood: ((_ : Chicken) => Egg) & ((_ : Cow) => Milk) =
    function (animal: any) {
        return animal.produce();
    };
```

Intersection type of two
function types.



Subtyping

- Intersection types

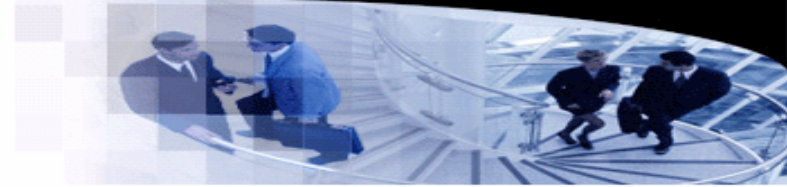
- Example in Typescript

```
var chicken = new Chicken();
var cow = new Cow();
var randomNumberGenerator = new RandomNumberGenerator();

console.log(chicken.produce()); //Egg { }
console.log(cow.produce()); //Milk { }
console.log(randomNumberGenerator.produce()); //0.2626353555444987

console.log(animalToFood(chicken)); //Egg { }
console.log(animalToFood(cow)); //Milk { }
//console.log(animalToFood(randomNumberGenerator)); //ERROR: Argument of type 'RandomNumberG

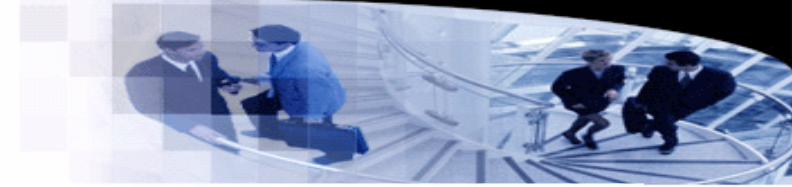
console.log(eatEgg(animalToFood(chicken))); //I ate an egg.
//console.log(eatEgg(animalToFood(cow))); //ERROR: Argument of type 'Milk' is not assignable
console.log(drinkMilk(animalToFood(cow))); //I drank some milk.
//console.log(drinkMilk(animalToFood(chicken))); //ERROR: Argument of type 'Egg' is not assi
```



Subtyping

• Intersection types

- Programming languages that support intersection types.



Language ↕	Actively developed ↕	Paradigm(s) ↕	Status ↕	Features ↕
C#	Yes ^[12]	• Functional • Imperative • Object-oriented	Under discussion ^[13]	?
Ceylon	Yes ^[14]	• Object-oriented	Supported ^[15]	• Type refinement • Interface composition • Subtyping in width
F#	Yes ^[16]	• Functional • Imperative • Object-oriented	Under discussion ^[17]	?
Flow	Yes ^[18]	• Functional • Imperative • Object-oriented • Scripting	Supported ^[19]	• Type refinement • Interface composition
Forsythe	No	• Imperative	Supported ^[20]	• Function type intersection • Distributive, co- and contravariant function type subtyping
Java	Yes ^[21]	• Imperative • Object-oriented	Supported ^[22]	• Type refinement • Interface composition • Subtyping in width
Scala	Yes ^[23]	• Functional • Imperative • Object-oriented	Supported ^{[24][25]}	• Type refinement • Trait composition • Subtyping in width
TypeScript	Yes ^[26]	• Functional • Imperative • Object-oriented • Scripting	Supported ^[5]	• Arbitrary type intersection • Interface composition • Subtyping in width and depth
Whiley	Yes ^[27]	• Functional • Imperative	Supported ^[28]	?